VESPAW

Tutorial

Hands-on examples with runnable, checked input decks

Mohsen Sotoudeh

Version of October 3, 2026

Contents

How to use this tutorial 1

1 Installation 2
- 1.1 One-step installation . . . 2
- 1.2 Building by hand . . . 2
- 1.3 Testing the build . . . 2
- 1.4 Running . . . 2

2 Input basics 3
- 2.1 Two files, blocks and keywords . . . 3
- 2.2 Choosing setups . . . 4
- 2.3 Convergence . . . 4

3 Example 1: a molecule (H_2O) 5
- 3.1 Input . . . 5
- 3.2 Results . . . 5
- 3.3 The relaxation . . . 5

4 Example 2: a semiconductor (Si) 7
- 4.1 Input . . . 7
- 4.2 Energy and band gap . . . 7
- 4.3 Equation of state . . . 7
- 4.4 Forces and stress . . . 7

5 Example 3: a metal (Al) 9

6 Example 4: a magnet (NiO) 10

7 Example 5: hybrid functionals (MgO) 11

8 Example 6: molecular dynamics (Si_8) 12

9 Choosing the optimizer and troubleshooting 13
- 9.1 Which optimizer? . . . 13
- 9.2 Troubleshooting . . . 13

10 Output and analysis 15
- 10.1 Files . . . 15
- 10.2 Density of states . . . 15
- 10.3 HDF5 and Python . . . 15

How to use this tutorial

Every example in this tutorial lives in a directory `examples/tutorial/NN_name/` and is run by

```
python3 doc/src/tutorial/check_examples.py             # all examples
python3 doc/src/tutorial/check_examples.py 02_si       # one example
python3 doc/src/tutorial/check_examples.py 02_si:scf   # one run of it
```

The input files in these directories are *templates*: words such as `@PSIDYN@` or `@LUNIT@` are placeholders. The file `case.json` lists the runs of an example and what each placeholder becomes in each run. The checker writes the actual inputs into `examples/tutorial/NN_name/work/RUN/` and runs them there. After a run you find in that directory exactly the input that was used, the report (`.prot`), the detailed protocol (`.log`) and all other output. Copy it to start your own calculation. VESPAW reports energies in eV, lengths in Å, forces in eV/Å and stress in GPa.

The checker also compares the results with the values quoted in this tutorial and reports `PASS` or `FAIL`. If the code changes so that an example no longer gives the numbers printed here, the checker says so. The numbers in this tutorial were obtained on an Apple M-series laptop (8 threads); timings are indicative.

The theory behind all examples is in the companion book *VESPAW: Theory and Methods* (`doc/theory.pdf`); every input keyword is described in the input manual `doc/manual.pdf`.

Chapter 1

Installation

1.1 One-step installation

```
git clone https://github.com/m-sotoudeh/VESPAW.git && cd VESPAW
./install.sh
export PATH="$HOME/.vespaw/bin:$PATH"
vespaw doctor
```

`install.sh` finds or installs what VESPAW needs (Fortran compiler, BLAS/LAPACK, FFTW, HDF5, MPI, Python) without administrator rights: on macOS with Homebrew it uses Homebrew, otherwise a private conda-forge environment in `~/.vespaw`; on clusters, load your modules and run `./install.sh --system`. It builds the serial and, if MPI works, the parallel version, and runs a short check. `vespaw doctor` reports the state of the installation.

1.2 Building by hand

The build uses a parameter file for the toolchain, in `arch/`:

```
./configure --with-parmfile=arch/parms.arm64     # Apple Silicon
make fast_new             # -> bin/<arch>/vespaw_fast.x
make fast_parallel_new    # -> bin/<arch>/vespaw_fast_mpi.x (MPI)
make tools scripts        # analysis tools and the vespaw dispatcher
```

Put `bin/<arch>/` on your `PATH`. Options and known build problems are listed in `doc/INSTALL.md`.

1.3 Testing the build

```
python3 testsuite/run.py                  # the full test suite
python3 doc/src/tutorial/check_examples.py 02_si:scf
```

The test suite runs every optimizer on a set of systems and checks energies, gaps, forces and stress against reference values and against each other; all cases should report `PASS`.

1.4 Running

A calculation needs a control file (`.cntl`) and a structure file (`.strc`) with the same stem:

```
vespaw si.cntl                            # serial, OpenMP threads
OMP_NUM_THREADS=8 vespaw si.cntl
OPENBLAS_NUM_THREADS=1 mpirun -np 4 vespaw si.cntl   # MPI
```

Chapter 2

Input basics

2.1 Two files, blocks and keywords

Input files consist of nested blocks. A block starts with `!NAME` and ends with `!END`; the file ends with `!EOB`. Keywords are written `KEY=value`; strings are in single quotes, logicals are `T` or `F`.

The control file (`.cntl`) says *what to do*:

- `!GENERIC`: number of steps `NSTEP`, time step `DT`, `START=T` (start from scratch) or `F` (continue from the restart file), `FORCES=T`, `STRESS=T` (print them at the end).
- `!FOURIER`: plane-wave cutoff `EPWPSI` in Rydberg and `CDUAL` (density cutoff = `CDUAL`×`EPWPSI`).
- `!DFT`: the functional, by name or number, e.g. `TYPE='PBE'` or `TYPE='PBEsol'` (the VASP tags `'PE'`, `'PS'`, ... also work; the list is in the manual); the local-hybrid block `!NTBO` inside it.
- `!PSIDYN`: how the electrons are optimized (`OPTIMIZER`, `MIXER`, convergence criteria).
- `!MERMIN`: electronic temperature (metals).
- `!RDYN`, `!CELL`: motion of atoms and cell.

The structure file (`.strc`) says *what the system is*:

- `!GENERIC LUNIT[AA]`: length unit of all coordinates, here Ångström (`LUNIT[AA]=1.0`).
- `!LATTICE T=`: the three lattice vectors (columns of T, in units of `LUNIT`).
- `!KPOINTS DIV=`: the k-point mesh; add `SYMMETRY=T` to reduce it to its irreducible wedge using the space group.
- `!OCCUPATIONS`: `NSPIN`, number of empty bands `EMPTY`, total spin `SPIN[HBAR]`, charge.
- `!SPECIES`: one block per element with the PAW setup (`!AUGMENT`) and, for hybrids, the local-orbital settings (`!NTBO`).
- `!ATOM NAME= R=`: atoms and positions.

Switching blocks off. Renaming a block (e.g. `!ORBPOT` → `!ORBPOT_OFF`) makes the program ignore it. `READ=F` inside a block also switches it off; this is easy to do by accident.

2.2 Choosing setups

The directory `potentials` contains a species block for every element. Copy the block of each element into the structure file. `potentials/AUDIT.md` lists which setups passed the quality checks, which are replaced, and how accurate they are in solids. Read it before choosing a setup for a production calculation. When a setup is built, the program prints a ghost test and a suggested cutoff (search the output for `GHOST TEST` and `SUGGESTED ENMAX`).

2.3 Convergence

Two parameters must always be converged: the cutoff `EPWPSI` and the k-point mesh. Converge the quantity you are interested in, not only the total energy. Example 1 shows that forces can require a much higher cutoff than the energy suggests, and Example 3 shows how the energy of a metal oscillates with the k-point mesh.

Chapter 3

Example 1: a molecule (H_2O)

Directory: `examples/01_h2o`. **You learn:** setting up a molecule in a box, converging the cutoff with forces, relaxing a geometry.

3.1 Input

The molecule sits in a cubic box of 10 Å with Γ-point sampling. It starts from a distorted geometry (O–H 1.00 Å, angle 110°). The control file: The run `scf` uses Davidson with the variational optimizer:

```
!PSIDYN OPTIMIZER='DAV' MIXER='VARIATIONAL' MIXGOSQ=0.05 SAFEORTHO=T STOP=T
```

`MIXGOSQ` sets the Kerker preconditioner. Its default (1.5) is made for solids; for molecules and slabs with a lot of vacuum use a small value (0.05–0.1), otherwise the variational optimizer stalls (see `doc/KNOWN_ISSUES.md`). Pulay mixing (`MIXER='PULAY'`) is a faster alternative for molecules.

3.2 Results

```
python3 doc/src/tutorial/check_examples.py 01_h2o
    scf: E = -471.41787880 eV, 249 it, gap 5.8369 eV
    relax: E = -471.48491560 eV, 446 it, gap 5.9828 eV
```

The relaxed geometry (in `work/relax/h2o.strc_out`) has O–H = 0.971 Å and H–O–H = 104.6°, in agreement with PBE (0.970 Å, 104.2°).

The cutoff lesson. At 30 Ry the same relaxation gives O–H = 1.007 Å and 102.0°, a bond 4% too long. At the PBE geometry the forces are 2.6 eV/Å at 30 Ry and 0.05 eV/Å at 60 Ry. Oxygen needs a high cutoff; converge it with forces, not only energies.

3.3 The relaxation

The run `relax` has two stages. Stage 1 converges the electrons (as `scf`); stage 2 continues from the restart file (`START=F`) and relaxes the atoms by damped dynamics of wave functions and atoms together:

```
!PSIDYN STOP=T FRIC=0.02 SAFEORTHO=T
  !AUTO FRIC(-)=0.3 FACT(-)=0.97 FRIC(+)=0.3 FACT(+)=1. MINFRIC=0.02 !END
!END
!RDYN STOP=T FRIC=0.05
```

```
  !AUTO FRIC(-)=0.3 FACT(-)=0.97 FRIC(+)=0.3 FACT(+)=1. MINFRIC=0.05 !END
!END
```

The `!AUTO` blocks ("autopilot") lower the friction while the energy decreases and stop the run when it has converged. `FORCES=T` prints the final forces (in eV/Å, also in section 07 of the `.prot` report):

```
FINAL FORCES [EV/AA]
FORCE O_1   ...
```

They are below 0.05 eV/Å at the end of the relaxation.

Chapter 4

Example 2: a semiconductor (Si)

Directory: `examples/02_si`. **You learn:** total energy, band gap, equation of state, forces and stress.

4.1 Input

The primitive cell of diamond Si has two atoms. The placeholders are the cutoff (`@EPW@`), the lattice scale (`@LUNIT@`), the position of the second atom (`@X2@`) and extra output (`@OUTPUT@`).

4.2 Energy and band gap

```
python3 doc/src/tutorial/check_examples.py 02_si:scf
    scf: E = -216.95911283 eV, 13 it, gap 0.7097 eV
```

The variational optimizer converges in 13 iterations. The Kohn–Sham gap of 0.71 eV on the $6 \times 6 \times 6$ mesh is the smallest gap between the k-points of the mesh (the true PBE gap of Si is about 0.6 eV, between Γ and a point near X that the mesh does not contain). Section 05b of the `.prot` report lists the band energies and occupations at every k-point.

4.3 Equation of state

The runs `eos_0.97` ... `eos_1.03` scale the lattice by `@LUNIT@` at 40 Ry:

a [Å]	E [eV]
5.305	−216.86290
5.387	−216.94416
5.469	−216.97061
5.551	−216.94984
5.633	−216.88874

A cubic fit of $E(V)$ gives $a_0 = 5.472$ Å and $B = 93$ GPa (all-electron PBE: 5.469 Å, 89 GPa; experiment: 5.431 Å). A Birch–Murnaghan fit is available with `vespaw_murnaghan.x`. Energy–volume curves need a converged cutoff: at fixed cutoff the number of plane waves changes with the volume (Pulay stress).

4.4 Forces and stress

The run `forces` displaces the second atom by 0.05 Å along x and sets `FORCES=T STRESS=T`:

```
FORCE SI1   0.6907426272   -0.0000290266   -0.0000008242
FORCE SI2  -0.6907157157    0.0000335846   -0.0000830316
STRESS      2.1972555    -0.0003638     0.0001797   ...
STRESS     -0.0003627     2.3652870    -2.0113901   ...
STRESS      0.0001752    -2.0113887     2.3651746   ...
PRESSURE[GPA]      2.3092391  VOLUME[AA^3]      40.02575175
```

The forces (eV/Å) are equal and opposite, as they must be. The stress is $\sigma = -(1/V)\,dE/d\epsilon$ in GPa, positive when the cell wants to expand. The displacement along x also produces a shear stress σ_{yz}: in diamond, an internal displacement couples to the shear strain. The test case `testsuite/si_forces` checks forces and stress against finite differences of the energy.

Chapter 5

Example 3: a metal (Al)

Directory: `examples/03_al`. **You learn:** electronic temperature and k-point convergence of a metal.

A metal has partially filled bands. `!MERMIN T[K]=1000.` gives the electrons a temperature, i.e. Fermi occupations; the program minimizes the free energy $A = E - TS$. The runs vary the k-point mesh:

mesh	A [eV], $T = 1000$ K
4^3	-57.02980
6^3	-57.36561
8^3	-57.29449
12^3	-57.30689
12^3, $T = 300$ K	-57.30277

The energy does not converge monotonically: the 6^3 mesh happens to be 59 meV too low. Only the difference between successive dense meshes tells you whether you are converged ($8^3 \to 12^3$: 12 meV). A lower temperature needs a denser mesh.

Chapter 6

Example 4: a magnet (NiO)

Directory: `examples/04_nio`. **You learn:** antiferromagnetic order, seeding the spin state, complete setups.

NiO orders antiferromagnetically: ferromagnetic (111) planes of Ni with alternating spin. The cell contains two Ni atoms, `NI1` (spin up) and `NI2` (spin down). The block

```
!ORBPOT
  !POT  ATOM='NI1' VALUE=+0.1 TYPE='D' S=1 RC=2. !END
  !POT  ATOM='NI2' VALUE=-0.1 TYPE='D' S=1 RC=2. !END
!END
```

adds a small spin-dependent potential on the Ni d orbitals that pushes the system into the desired spin order. It is used only in the first stage and then switched off by renaming it (`!ORBPOT_OFF`); the placeholder `@ORBPOT@` does exactly this.

```
python3 doc/src/tutorial/check_examples.py 04_nio
    dav_var: E = -3030.79719315 eV, 274 it, gap 1.4979 eV
    dav_coldseed: E = -3030.79713862 eV, 82 it, gap 1.5002 eV
```

`dav_var` seeds the spin order with friction dynamics and the orbital potential, then converges with Davidson; `dav_coldseed` seeds and converges with Davidson directly. Both reach the same state. The `.prot` report lists the magnetic moments: about $\pm 1.3\ \mu_B$ in the Ni spheres, zero in total. The PBE gap of 1.5 eV is far below the experimental 4.3 eV; see Example 5 for hybrids.

Complete setups. The Ni setup has two partial waves per angular momentum (`NPRO=2 2 2`). With a single s projector the calculation can collapse into an unphysical state far below the ground state; the setup generator warns about this (`INCOMPLETE SETUP`).

Chapter 7

Example 5: hybrid functionals (MgO)

Directory: `examples/05_mgo_hybrid`. **You learn:** the local hybrids `PBE0r` and `PBE-SL0r`.

The placeholder `@NTBO@` is empty for PBE, and for the hybrids

```
# PBE0r
!NTBO MODUS='HYBRID' HFWEIGHT=0.25 K2=-0.25 SCALERCUT=2. !END
# PBE-SL0r
!NTBO MODUS='HYBRID' HFWEIGHT=0.25 DCTYPE='SCANL' K2=-0.25 SCALERCUT=2. !END
```

`HFWEIGHT` is the fraction of exact exchange. `PBE0r` replaces part of the onsite PBE exchange of the local orbitals by exact exchange; `PBE-SL0r` in addition uses SCAN-L instead of PBE for that onsite exchange (`DCTYPE='SCANL'`), with PBE everywhere else. A third local hybrid, `SCAN0r`, uses SCAN everywhere (`TYPE='SCAN' TSCANDELOC=T`); it is still experimental. The local orbitals are defined per species in the structure file: `NOFL` gives the number of local orbitals per angular momentum, `LHFWEIGHT` a species-specific exchange fraction, `CV=T` includes core–valence exchange.

functional	gap [eV]
PBE	4.326
PBE0r	4.034
PBE-SL0r	4.116

The local hybrids act only on the orbitals of the atoms (onsite). For MgO, a closed-shell oxide, the correction is small; for oxides with open d shells (NiO, manganites) it is large. Take band gaps of hybrid calculations from Davidson, not from RMM-DIIS, which can miss the lowest empty band with hybrids.

Chapter 8

Example 6: molecular dynamics (Si_8)

Directory: `examples/06_md`. **You learn:** Car–Parrinello molecular dynamics with a thermostat, and a pitfall of small cells.

Stage 1 converges the electrons. Stage 2 runs Car–Parrinello dynamics without friction on the wave functions (`FRIC=0.`), with time step `DT=10.0` a.u. (0.24 fs), random initial velocities for 2000 K (`RANDOM[K]`; equipartition quickly halves the temperature) and a Nosé thermostat at 1000 K:

```
!RDYN STOP=T RANDOM[K]=2000.
  !THERMOSTAT T[K]=1000. FREQ[THZ]=10. !END
!END
```

The detailed protocol (`.log`) prints a line per step, in atomic units (Hartree):

```
    NFI  T[PSEC] T[K]  EKIN(PSI)       E(RHO)        ECONS   ANNEE   ANNER
!>   100   0.02419   696   0.00229    -31.82720    -31.77515 0.00000 0.00143
!>   500   0.12094  1160   0.00394    -31.83888    -31.77518 0.00000 0.00307
```

with time, ionic temperature, fictitious kinetic energy of the wave functions `EKIN(PSI)`, the potential energy `E(RHO)` and the conserved energy `ECONS`. `ECONS` must stay constant; here it changes by 3×10^{-5} H (0.8 meV) in 400 steps. `EKIN(PSI)` must stay small compared with the ionic kinetic energy (adiabaticity).

The small-cell pitfall. With Γ-point sampling only, the same run lets the energy fall by 6.8 eV below that of diamond: in an 8-atom cell sampled at Γ alone, distorted structures are lower in energy than diamond. With a $2 \times 2 \times 2$ mesh diamond is correctly lower by 4.4 eV. For MD, use a cell large enough for Γ-point sampling, or a k-point mesh.

Chapter 9

Choosing the optimizer and troubleshooting

9.1 Which optimizer?

`OPTIMIZER='DAV'`	block Davidson. Robust from any start; the reference. Default choice for small and medium systems.
`OPTIMIZER='RMM'`	RMM-DIIS with a Davidson warm-up. Fastest for large systems (64 atoms and more). Not for band gaps with hybrids.
`OPTIMIZER='CG'`	conjugate gradients, band by band. Reliable, slower for many bands.
dynamics (default)	damped Car–Parrinello dynamics (`FRIC`, `!AUTO`). Slow but very robust; required for molecular dynamics and convenient for relaxations.
`OPTIMIZER='EDFT'`	direct joint minimization of orbitals and occupations (Route 2). Experimental.

With the eigensolvers the mixer is `MIXER='VARIATIONAL'` (default): every accepted energy is an upper bound of the ground-state energy and decreases monotonically. `MIXER='PULAY'` and `'ANDERSON'` are classical density mixing. Convergence is controlled by `EDIFF` (energy change, default 2.7×10^{-5} eV) and `MIXTOL` (residual).

9.2 Troubleshooting

The SCF does not converge.

- Molecules and slabs: reduce `MIXGOSQ` to 0.05–0.1.
- Metals: set an electronic temperature (`!MERMIN`).
- Magnetic systems: seed the spin order (`!ORBPOT`, Example 4), or start with a few steps of damped dynamics.
- Compare with damped dynamics. If the variational optimizer finds an energy *below* the dynamics result without converging, the potential is not the derivative of the energy; that is a bug worth reporting.

The energy is far too low. A collapse into an unphysical state: check the setup output for `INCOMPLETE SETUP` or `GHOST`, use complete setups (two projectors per angular momentum), and look for very deep eigenvalues.

Wrong structure. Converge the cutoff with forces (Example 1) and the k-points (Example 3); for $3d$ metals read the accuracy notes in `potentials/AUDIT.md`.

Orthogonalization fails at the start. `LOOP FOR ORTHOGONALIZATION IS NOT CONVERGED` in damped dynamics from random wave functions: start with a few Davidson steps, or with `SAFEORTHO=F`.

Chapter 10

Output and analysis

10.1 Files

`.prot`	report: input, system, model, SCF history, energies, k-points and occupations, atomic charges and moments, forces, stress, timing
`.log`	detailed protocol: setup construction, per-step reports
standard output	live table of iterations and the final energy
`results.h5`	numerical results (with `!HDF5OUT`)
`.rstrt`	restart file (wave functions, positions) for `START=F`
`.strc_out`	final structure
`.pdos`	projections for the density of states
`.tra`	trajectories (MD, relaxation)

10.2 Density of states

The DOS tool reads the `.pdos` file. For Si (`examples/02_si/si.dcntl`):

```
cd examples/tutorial/02_si/work/scf
vespaw_dos.x ../../si.dcntl
```

writes `dos_total.dos`, `dos_si_s.dos` and `dos_si_p.dos` (energy in eV, DOS, integrated DOS). Weights can be atoms and angular momenta (`TYPE='S','P','D'`) or single orbitals (`!ORB`, e.g. e_g and t_{2g}); see `doc/manual.pdf`, "The Density of States Analysis Tool".

10.3 HDF5 and Python

With `!HDF5OUT READ=T FILE='results.h5' !END` in the `!CONTROL` block the program writes energy, forces, eigenvalues and provenance to an HDF5 file. The Python package reads it:

```
pip install -e python/
export PATH="$HOME/.vespaw/bin:$PATH"     # after install.sh

from vespaw import Calculation
r = Calculation("si.cntl").run()
print(r.energy, r.gap, r.forces)   # eV, eV, eV/Angstrom
```

For the Si example this prints $E = -216.95911$ eV and a gap of 0.7097 eV. A YAML input front end is described in `doc/USAGE.md`.